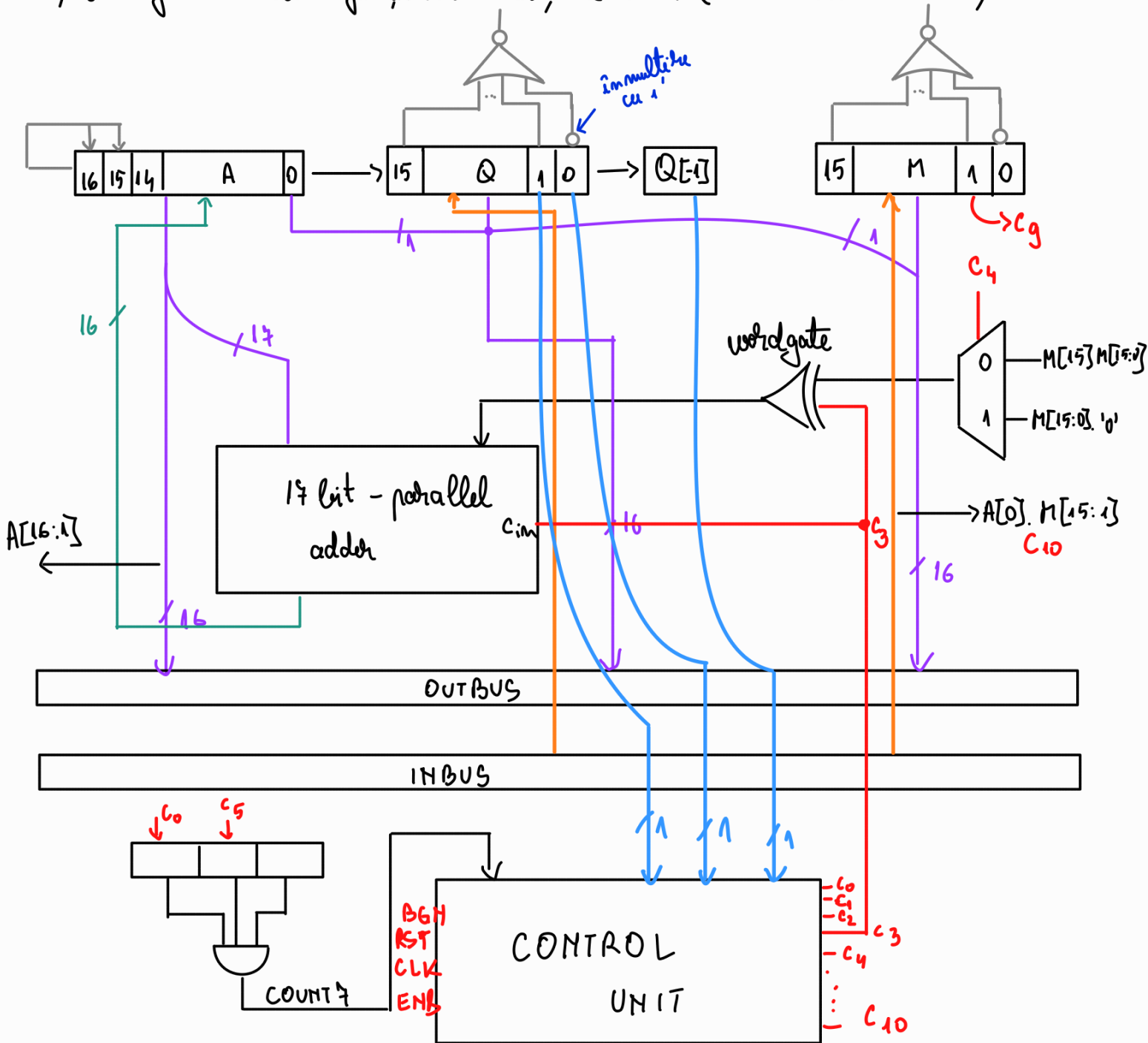


Examen / 2023 - 2024

① Booth - radix 4 $\rightarrow$ nr. fracționare pe 16 biți și să îl trateze în mod distinct situațiile de înmulțire cu „1” (rezultatul este egal cu celălalt operand, exprimat pe 32 de biți)

a) Schema bloc hardware pe platforma Hayes (cu semnale de control)

b) Ordinograma ce descrie funcționarea unității de control (cu semnale de control)



2. $CR = 5 \text{ GHz}$

Instn type	CPI ^{→ cc}	ic
AL	5	5000
MOV	3	10 000
L/S	3	3000
Others	2	2000

2 echipe

- hardware
 - opt. set instn
 - AL → de la 5 la 3
 - L/S → de la 3 la 2
 - se degradează CCT cu 5%
- software
 - opt. compiler
 - MOV → 20% reduce ic
 - other cu 10%

a) CPU_{TIME} b) MIPS

$$CCT_{old} = \frac{1}{5 \text{ GHz}} = 0,2 \text{ } \mu\text{s}$$

$$CCT_{new} = 0,2 \cdot \frac{105}{100} = 0,21 \text{ } \mu\text{s} \rightarrow \text{pt hardware}$$

$$ic_{MOV_{sw}} = \frac{80}{100} \cdot 10\,000 = 8000$$

$$ic_{other_{sw}} = \frac{90}{100} \cdot 2000 = 1800$$

$$CPU_{TIME} = ic \cdot CPI \cdot CCT = (5 \cdot 5000 + 3 \cdot 10000 + 3 \cdot 3000 + 2 \cdot 2000) \cdot 0,2 =$$

$$= 13,6 \text{ } \mu\text{s}$$

$$CPU_{TIME_{HW}} = (3 \cdot 5000 + 3 \cdot 10000 + 3 \cdot 3000 + 2 \cdot 2000) \cdot 0,21 = 11,55 \mu s$$

$$CPU_{TIME_{SW}} = (5 \cdot 5000 + 3 \cdot 8000 + 3 \cdot 3000 + 2 \cdot 1800) \cdot 0,2 = 12,32 \mu s$$

$\Rightarrow$ HW team is better

$$b) \quad ic_{HW} = 5000 + 10000 + 3000 + 2000 = 20000$$

$$ic_{SW} = 5000 + 8000 + 3000 + 1800 = 17800$$

$$MIPS_{HW} = \frac{ic}{CPU_{TIME} \cdot 10^6} = \frac{20000}{11,55 \mu s \cdot 10^6} = 1731,6$$

$\downarrow$
 10^{-6}

$$MIPS_{SW} = \frac{17800}{12,32 \mu s \cdot 10^6} = 1444,8$$

} HW is better

③ Avem un computer load/store cu un sistem cache ce se caracterizează prin următoarele valori empirice atunci când rula un anumit program: 10% Data Miss Rate, 8% Instruction Miss Rate și 20% instrucțiuni load/store. În orice moment, 20% dintr-un blocuri sunt modificate (i.e., dirty). Programul determină un CPI de 2 clock cycles dacă cache-ul este perfect și costă din 10 000 de instrucțiuni în cod mașină. Calculatorul are o frecvență a clock-ului de 4.5 GHz. Penalitatea de miss este determinată de faptul că un access la BUS ^{introduce} ~~introduce~~ 10 cicluri de clock, lățimea BUS-ului este de 4 cuvinte, block-ul are 16 cuvinte, iar cache-ul implementează o politică Write Back cu Write Allocate în cazul unui miss. Calculați:

(1P) a) ~~TRAT~~, știind că $t_{ac} = 2 \text{ ns}$.

(1P) b) CPI final.

$$\begin{aligned}
 \text{3 a) Read Miss Penalty} &= \underbrace{\frac{16 \text{ words}}{4 \text{ words}}}_{\text{Bus}} \cdot \underbrace{0,2}_{\text{1 block dirty}} \text{ Bus writes} + \\
 &\quad \underbrace{\frac{16 \text{ words}}{4 \text{ words}}}_{\text{allocate}} \text{ Bus Reads} = 4 \cdot 0,2 + 4 = 4 \cdot 1,2 = 4,8 \text{ Bus accesses}
 \end{aligned}$$

$$\Rightarrow \text{Miss Penalty} = 4,8 \cdot 10 = 48 \text{ C.C.}$$

$$\text{CCT} = \frac{1}{4,5 \cdot 10^{-9}} = 0,2222 \mu\text{s}$$

$$\begin{aligned}
 \text{Misses Per Instruction} &= \text{Miss Rate} \cdot \text{Mem. accesses per instruction} \\
 \text{altă f.} &= 1 \times \text{IMR} + \underbrace{0,2 \cdot \text{BMR}}_{\nearrow 4/5} = 0,08 + 0,2 \cdot 0,1 = 0,1
 \end{aligned}$$

$$\text{Miss Rate} = \frac{0,1}{1,2} = 0,0833$$

mem access

$$AMAT = 0,4444 \mu s + \frac{0,833 \cdot 48CC}{CCT \cdot t_{ac} + MR \cdot MP \cdot CCT} \cdot 0,2222 = 1,3328 \mu s$$

$$b) CPU_{TIME} = IC \cdot (CPI_{ideal} + MPi \cdot MP) \cdot CCT =$$

$$= 10\,000 (2 + 0,1 \cdot 48) \cdot 0,2222 = 15,1096 \mu s$$

④ Se dă un sistem de memorie, cu spațiul virtual de dimensiune I
 ③P 256 TiB, spațiu fizic de 256 KiB și adresa la nivel de byte. Cursăntul
 este pe 64 de biți, block-ul are 64 de cursănti. O pagină de memorie
 are dimensiunea de 64 KiB, iar dimensiunea de date a cache-ului
 (SA pe 2 căi) este de 256 KiB. Cache-ul este Write Back, cu
 Write Allocate în caz de Write Miss și are o politică de înlocuire
 random. TLB-ul are 64 de intrări și folosește înlocuire directă.
 Se cere:

- ①P a) Formatul adresă de memorie virtuală respectiv fizică
 ①P b) Dimensiunea totală a memoriei cache și a TLB-ului.
 ①P c) Schema bloc a mapeării adresă fizică-adresă virtuală
 ce conține memoria cache și TLB-ul (ca în curs).

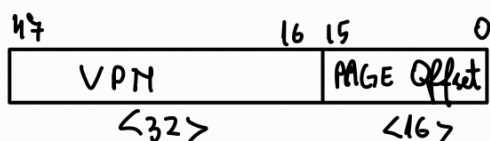
a) $VMSize = 2^8 \times 2^{40} B = 2^{48} B$ $Page = 64 KiB = 2^{16} B$
 $PMSize = 2^{38} B$ $TLB entries = 2^6 \rightarrow TLB index$
 1 word = $2^3 B$ Word Offset
 1 block = $2^6 words = 2^6 \cdot 2^3 B = 2^9 B$ Block Offset

a) $VM size = 2^8 \cdot 2^{40} = 2^{48} B \rightarrow$ Nr. biți VPM
 $PM size = 2^9 \cdot 2^{30} = 2^{39} B$
 $page = 64 KiB = 2^6 \cdot 2^{10} = 2^{16} B \rightarrow$ page offset = 16

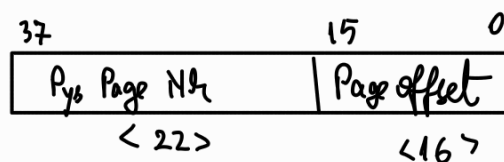
1 word = 64 biți = $8 B = 2^3 B \Rightarrow$ W.O. = 3

1 block = 64 words = $2^6 \Rightarrow$ B.O. = 6

TLB entries = 64 entries = $2^6 \Rightarrow$ TLB index = 6



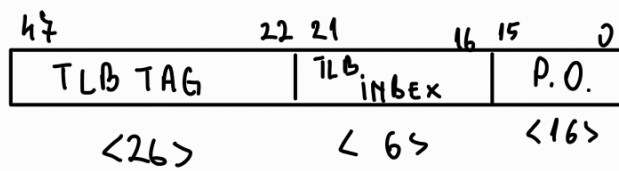
virtual memory



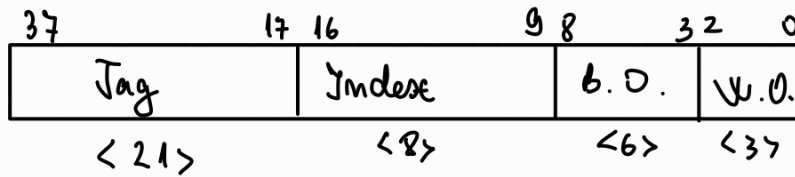
physical memory

cache data size = $256 KiB = 2^8 \cdot 2^{10} = 2^{18} B = \frac{2^{18} B}{2^6 \cdot 2^3 B} = 2^9$ blocks

bank size = $\frac{2^9 \text{ blocks}}{2^1 \text{ S.A.}} = 2^8 \text{ blocks/bank}$
 index



V.M.



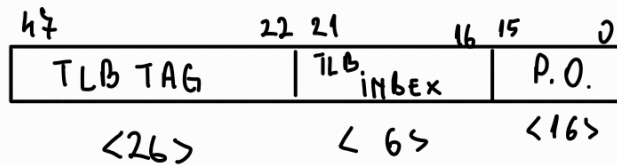
P.M.

b)

$$\begin{aligned}
 \text{Cache size} &= 2^1 \cdot 2^8 \cdot (2^9 \text{ B} + \underbrace{1 \text{ b} + 1 \text{ b} + 21 \text{ b}}_{\approx 3 \text{ B}}) = \\
 &= 2^9 \cdot 2^9 \text{ B} + 2^9 \cdot 3 = 257,5 \text{ KiB}
 \end{aligned}$$

$$\text{TLB size} = 2^6 (1 \text{ b} + 26 \text{ b} + 22 \text{ b}) = 2^6 \cdot 49 \text{ b} = 2^6 \cdot 2^3 \text{ B} = 2^9 \text{ B} = 0,5 \text{ KiB}$$

c)



6. A.

```
for (i=0, p=0, m=0; i < max; i++) {  
    if (i%2 == 0) {  
        p++  
    }  
    else m++  
}
```

$r_0 \rightarrow \text{adr}$

$r_1 \rightarrow i$

$r_2 \rightarrow p$

$r_3 \rightarrow m$

$r_4 \rightarrow \text{max}$

```
init:    MOV    r1, #1  
         MOV    r2, #0  
         MOV    r3, #0  
         ABR    r0, max  
         LBR    r3, [r0]
```

```
loop:    CMP    r1, r4     $\rightarrow$  comp. i cu max  
         BGE    end  
         ANB    r5, r2, #1  $\leftarrow$  te uite pe ultimul bit  
         CMP    r5, #0  
         ABBEQ  r2, r2, #1  
         ABBNE  r3, r3, #1  
         ABB    r1, r1, #1
```

loop

```
end      ABR    r0, p  
         STR    r2, [r0]
```

```
ABR    r0, m  
STR    r3, [r0]
```

```

(13) for (i = 0, m = 0; i < max; i++) {
    m = m + i;
    if (m > 2 * max) {
        m = 0;
    }
}

```

$r_0 \rightarrow \text{addr}$

$r_1 \rightarrow i$

$r_2 \rightarrow m$

$r_3 \rightarrow \text{max}$

init: MOV $r_1, \#0$

MOV $r_2, \#0$

ABR r_0, max

LBR $r_3, [r_0]$

loop: CMP r_1, r_3

BGE end

ABB r_2, r_2, r_1

ABB r_4, r_3, r_3

CMP r_2, r_4

MOVGT $r_2, \#0$

ABB $r_1, r_1, \#1$

B loop

$\nearrow \text{shift } m$
 LSL $r_4, r_3, \#1$